

```
C:\Users\User\source\repos\A... x + ~
=====
ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ API
AgroControlAPI
=====

?? Тест 1: Успешная авторизация... ? PASS
?? Тест 2: Неверный пароль (негативный)... ? PASS (ожидаемая ошибка 401)
?? Тест 3: Получение активных партий... ? PASS
?? Тест 4: Получение программы партии (batchId=1)... ? PASS
? Тест 5: Начало шага (stepId=13)... ? PASS
? Тест 6: Завершение шага в норме (stepId=13)... ? PASS
? Тест 7: Завершение шага с отклонением (stepId=13)... ? PASS
?? Тест 8: Отправка сообщения о проблеме (batchId=1, stepId=13)... ? PASS
?? Тест 9: Получение журнала партии (batchId=1)... ? PASS
?? Тест 10: Получение уведомлений... ? PASS

=====
РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ
? Пройдено: 10
? Не пройдено: 0
?? Всего: 10
=====

Нажмите любую клавишу для выхода...
```

using System;

using System.Net.Http;

using System.Net.Http.Headers;

using System.Text;

using System.Threading.Tasks;

using Newtonsoft.Json.Linq;

namespace AgroControlAPI.IntegrationTests

{

class Program

{

private static readonly HttpClient _client = new HttpClient();

private static string _token = "";

private static readonly string _baseUrl = "http://localhost:5134/api";

// ===== ВАШИ РЕАЛЬНЫЕ ДАННЫЕ =====

private static readonly string _username = "operator.zavodov";

private static readonly string _password = "hash123"; // хеш или реальный пароль?

private static readonly long _batchId = 1; // B-2401-01

private static readonly long _stepId = 13; // Экструзия (pending)

private static readonly long _operatorId = 3; // operator.zavodov

```
static async Task Main(string[] args)
{
    Console.WriteLine("=====");
    Console.WriteLine("ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ API");
    Console.WriteLine("AgroControlAPI");
    Console.WriteLine("=====\\n");

    int passed = 0;
    int failed = 0;

    // Тест 1: Авторизация (позитивный)
    if (await TestLogin()) passed++; else failed++;

    // Тест 2: Авторизация с неверным паролем (негативный)
    if (await TestLoginInvalidPassword()) passed++; else failed++;

    // Тест 3: Получение активных партий
    if (await TestGetActiveBatches()) passed++; else failed++;

    // Тест 4: Получение программы партии
    if (await TestGetBatchProgram()) passed++; else failed++;

    // Тест 5: Начало шага
    if (await TestStartStep()) passed++; else failed++;

    // Тест 6: Завершение шага в норме
    if (await TestCompleteStepNormal()) passed++; else failed++;

    // Тест 7: Завершение шага с отклонением (негативный)
    if (await TestCompleteStepDeviation()) passed++; else failed++;
}
```

```

// Тест 8: Отправка сообщения о проблеме
if (await TestReportProblem()) passed++; else failed++;

// Тест 9: Получение журнала партии
if (await TestGetBatchJournal()) passed++; else failed++;

// Тест 10: Получение уведомлений
if (await TestGetNotifications()) passed++; else failed++;

Console.WriteLine("\n=====");
Console.WriteLine($"РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ");
Console.WriteLine($"✅ Пройдено: {passed}");
Console.WriteLine($"❌ Не пройдено: {failed}");
Console.WriteLine($"📊 Всего: {passed + failed}");
Console.WriteLine("=====");

Console.WriteLine("\nНажмите любую клавишу для выхода...");
Console.ReadKey();
}

// ===== ТЕСТ 1: Успешная авторизация =====
static async Task<bool> TestLogin()
{
    Console.WriteLine("🔑 Тест 1: Успешная авторизация... ");
    try
    {
        var data = new { username = _username, password = _password };
        var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(data),
            Encoding.UTF8, "application/json");
        var response = await _client.PostAsync($"({_baseUrl})/Auth/login", content);
        var responseBody = await response.Content.ReadAsStringAsync();
    }
}

```

```

        if (response.IsSuccessStatusCode)
        {
            var json = JObject.Parse(responseBody);
            _token = json["token"]?.ToString();
            if (!string.IsNullOrEmpty(_token))
            {
                _client.DefaultRequestHeaders.Authorization = new
System.Net.Http.Headers.AuthenticationHeaderValue("Bearer", _token);

                Console.WriteLine(" ✅ PASS");
                return true;
            }
        }

        Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");
        return false;
    }

    catch (Exception ex)
    {
        Console.WriteLine($" ❌ FAIL ({ex.Message})");
        return false;
    }
}

// ===== ТЕСТ 2: Неверный пароль =====

static async Task<bool> TestLoginInvalidPassword()
{
    Console.WriteLine(" 🚫 Тест 2: Неверный пароль (негативный)... ");

    try
    {
        var data = new { username = _username, password = "wrongpassword" };
        var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(data),
Encoding.UTF8, "application/json");

        var response = await _client.PostAsync($"{_baseUrl}/Auth/login", content);
    }
}

```

```

        if (response.StatusCode == System.Net.HttpStatusCode.Unauthorized)
        {
            Console.WriteLine("  PASS (ожидаемая ошибка 401)");
            return true;
        }

        Console.WriteLine("  FAIL");
        return false;
    }

    catch (Exception ex)
    {
        Console.WriteLine($"  FAIL ({ex.Message})");
        return false;
    }
}

// ===== ТЕСТ 3: Получение активных партий =====

static async Task<bool> TestGetActiveBatches()
{
    Console.WriteLine("  Тест 3: Получение активных партий... ");
    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine("  FAIL (нет токена)");
            return false;
        }

        var response = await _client.GetAsync($"{_baseUrl}/Production/active-batches");

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine("  PASS");
        }
    }
}

```

```

        return true;
    }

    Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");
    return false;
}

catch (Exception ex)
{
    Console.WriteLine($" ❌ FAIL ({ex.Message})");
    return false;
}
}

```

// ===== ТЕСТ 4: Получение программы партии =====

```

static async Task<bool> TestGetBatchProgram()
{
    Console.WriteLine($" 📄 Тест 4: Получение программы партии (batchId={_batchId})... ");
    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine($" ❌ FAIL (нет токена)");
            return false;
        }

        var response = await _client.GetAsync($" {_baseUrl}/Production/batch/{_batchId}/program");

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine($" ✅ PASS");
            return true;
        }

        Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");
    }
}

```

```

        return false;
    }
    catch (Exception ex)
    {
        Console.WriteLine($" ❌ FAIL ({ex.Message})");
        return false;
    }
}

// ===== ТЕСТ 5: Начало шага =====

static async Task<bool> TestStartStep()
{
    Console.WriteLine($" ▶ Тест 5: Начало шага (stepId={_stepId})... ");
    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine(" ❌ FAIL (нет токена)");
            return false;
        }

        var data = new { operatorId = _operatorId };
        var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(data),
            Encoding.UTF8, "application/json");

        var response = await _client.PostAsync($" {_baseUrl}/Production/step/{_stepId}/start",
            content);

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine(" ✅ PASS");
            return true;
        }

        Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");
    }
}

```

```

        return false;
    }
    catch (Exception ex)
    {
        Console.WriteLine($" ❌ FAIL ({ex.Message})");
        return false;
    }
}

```

// ===== ТЕСТ 6: Завершение шага в норме =====

```
static async Task<bool> TestCompleteStepNormal()
```

```

{
    Console.WriteLine($"✓ Тест 6: Завершение шага в норме (stepId={_stepId})... ");
    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine(" ❌ FAIL (нет токена)");
            return false;
        }
    }
}

```

```

        var data = new { actualValue = 82, actualDurationMin = 30, comment = "", operatorId =
        _operatorId };

```

```

        var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(data),
        Encoding.UTF8, "application/json");

```

```

        var response = await _client.PostAsync($" {_baseUrl}/Production/step/{_stepId}/complete",
        content);

```

```

        if (response.IsSuccessStatusCode)

```

```

        {
            Console.WriteLine(" ✅ PASS");
            return true;
        }

```

```

        Console.WriteLine($" ❌ FAIL {(int)response.StatusCode}");
        return false;
    }
    catch (Exception ex)
    {
        Console.WriteLine($" ❌ FAIL {ex.Message}");
        return false;
    }
}

// ===== ТЕСТ 7: Завершение шага с отклонением =====
static async Task<bool> TestCompleteStepDeviation()
{
    Console.WriteLine($" ⚠ Тест 7: Завершение шага с отклонением (stepId={_stepId})... ");
    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine($" ❌ FAIL (нет токена)");
            return false;
        }

        var data = new { actualValue = 95, actualDurationMin = 30, comment = "", operatorId =
        _operatorId };

        var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(data),
        Encoding.UTF8, "application/json");

        var response = await _client.PostAsync($" {_baseUrl}/Production/step/{_stepId}/complete",
        content);

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine($" ✅ PASS");
            return true;
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($" ❌ FAIL {ex.Message}");
        return false;
    }
}

```

```

    }

    Console.WriteLine($" ❌ FAIL {(int)response.StatusCode}");

    return false;
}

catch (Exception ex)
{
    Console.WriteLine($" ❌ FAIL {ex.Message}");

    return false;
}
}

// ===== ТЕСТ 8: Отправка сообщения о проблеме =====

static async Task<bool> TestReportProblem()
{
    Console.WriteLine($" 🚩 Тест 8: Отправка сообщения о проблеме (batchId={_batchId},
stepId={_stepId})... ");

    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine($" ❌ FAIL (нет токена)");

            return false;
        }

        var data = new { batchId = _batchId, stepId = _stepId, problemType = "Оборудование",
description = "Тестовое сообщение", severity = "high", operatorId = _operatorId };

        var content = new StringContent(Newtonsoft.Json.JsonConvert.SerializeObject(data),
Encoding.UTF8, "application/json");

        var response = await _client.PostAsync($" {_baseUrl}/Production/report-problem", content);

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine($" ✅ PASS");
        }
    }
}

```

```

        return true;
    }

    Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");

    return false;
}

catch (Exception ex)
{
    Console.WriteLine($" ❌ FAIL ({ex.Message})");

    return false;
}
}

```

// ===== ТЕСТ 9: Получение журнала партии =====

```

static async Task<bool> TestGetBatchJournal()
{
    Console.WriteLine($" 📄 Тест 9: Получение журнала партии (batchId={_batchId})... ");

    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine($" ❌ FAIL (нет токена)");

            return false;
        }

        var response = await _client.GetAsync($" {_baseUrl}/Production/batch/{_batchId}/journal");

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine($" ✅ PASS");

            return true;
        }

        Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");
    }
}

```

```

        return false;
    }
    catch (Exception ex)
    {
        Console.WriteLine($" ❌ FAIL ({ex.Message})");
        return false;
    }
}

// ===== ТЕСТ 10: Получение уведомлений =====
static async Task<bool> TestGetNotifications()
{
    Console.WriteLine(" 🛎 Тест 10: Получение уведомлений... ");
    try
    {
        if (string.IsNullOrEmpty(_token))
        {
            Console.WriteLine(" ❌ FAIL (нет токена)");
            return false;
        }

        var response = await _client.GetAsync($"_{baseUrl}/Notifications");

        if (response.IsSuccessStatusCode)
        {
            Console.WriteLine(" ✅ PASS");
            return true;
        }

        Console.WriteLine($" ❌ FAIL ({(int)response.StatusCode})");
        return false;
    }
    catch (Exception ex)

```

```
{  
    Console.WriteLine($" ❌ FAIL ({ex.Message})");  
    return false;  
}  
}  
}  
}
```